



# 구조 이성질체 탐색 및 안정성 평가 알고리즘 개발

## 1. 탐구 동기

유기 화합물은 동일한 분자식을 가지면서도 원자의 배열이 달라 서로 다른 성질을 나타내는 이성질체를 형성한다. 이성질체의 종류와 안정성을 예측하는 것은 신약 개발, 재료 과학, 촉매 설계 등 다양한 분야에서 중요하다.

기존에는 이성질체를 탐색할 때 화학자의 직관과 경험에 의존하거나, 제한적인 규칙 기반 시스템을 사용했다. 하지만 탄소 수가 증가하면 가능한 이성질체의 개수가 기하급수적으로 증가하여 직접 모든 경우를 빠짐없이 찾기 어렵다.

본 탐구에서는 **그래프 이론**과 **양자화학 계산**을 결합하여, 주어진 분자식으로부터 가능한 모든 구조 이성질체를 체계적으로 생성하고, **밀도범함수 이론(DFT)**을 통해 각 이성질체의 안정성을 정량적으로 평가하는 알고리즘을 개발해 보았다.

## 2. 이론적 배경

### 이성질체의 정의와 분류

이성질체란 분자식이 같지만 구조가 달라서 물리적, 화학적 성질이 다른 화합물로, 원자의 결합 순서가 다른 **구조 이성질체**와 입체적인 배열이 다른 **입체 이성질체**로 나눌 수 있다.

구조 이성질체에는 탄소 골격 구조가 서로 다른 **골격 이성질체**, 작용기 위치가 다른 **위치 이성질체**, 서로 다른 작용기를 가진 **작용기 이성질체**가 있고, 입체 이성질체에는 서로 거울상 관계에 있는 **거울상 이성질체** 또는 **광학 이성질체**, 일부만 입체 이성질체 관계를 가지고 있는 **부분입체 이성질체**, 이중 결합을 기준으로 원자나 작용기의 공간 배열이 다른 **기하 이성질체**가 있다.

### 혼성 궤도 함수 (Hybrid Orbitals)

**궤도 함수**란 원자핵 주위의 전자 1개에 대해 전자를 찾을 확률을 나타내는 양자역학적 파동 함수로 **오비탈**이라고도 불린다.  $s, p, d, f$  오비탈 등이 있으며, 각 오비탈은 고유한 모양과 에너지 준위를 가진다.

혼성 궤도 함수란 공유 결합을 형성하기 위해 한 원자의 서로 다른 원자 궤도 함수를 섞어 만든 새로운 궤도 함수이다. 이때 쌍극자 반발을 최소화하고 결합 방향성과 결합 수를 최적으로 배치하게 된다. 혼성화는 양자 역학적으로 엄밀한 이론은 아니지만, 결합이나 여러 화합물의 구조를 설명하는 데 매우 효과적으로 사용할 수 있다. 하지만 결합 방향과 기하적인 구조로 설명하는 이론이기 때문에 각 궤도의 에너지를 정량적으로 분석할 수 없고, 분자의 전체 에너지를 계산할 수 없다는 한계점이 존재한다.

### 분자 궤도 이론 (Molecular Orbital Theory)

**분자 궤도 함수**란 분자 내에서 전자의 거동을 기술하는 수학적 함수로써, 특정 위치에서의 전자 발견 확률과 같은 화학적, 물리적 특성을 계산하기 위해서 사용된다. 분자 오비탈을 통해 분자 내 하나 또는 한쌍의 전자에 대해 에너지나 분자의 전자배치에 대한 정보를 얻을 수 있다.

분자 오비탈은 주로 **원자 오비탈의 선형결합**을 통해 나타내게 된다. 이때 분자 궤도에서 전자는 분자 전체에 비편재화되어 표현된다.

분자 궤도에는 **결합성 궤도**, **반결합성 궤도**, **비결합성 궤도**가 있다. 결합성 궤도는 원자 궤도들이 같은 위상으로 겹쳐져 보강 간섭이 일어나 형성된다. 이 경우 에너지 준위가 낮아져 분자가 더 안정한 상태가 된다. 반결합성 궤도는 원자 궤도들이 반대 위상으로 겹쳐져 상쇄 간섭을 일어킬 때 형성된다. 이 경우 에너지 준위가 높아져 분자가 불안정해지게 된다. 비결합성 궤도는 결합에 직접적으로 참여하지 않고, 원자 궤도의 성질을 그대로 유지하는 궤도로 결합 형성에 기여하지 않기 때문에 에너지 변화가 거의 없다. 비결합성 궤도는 고립 전자쌍에서 나타난다.

## HOMO-LUMO Gap

**HOMO(Highest Occupied Molecular Orbital)**는 전자가 채워진 가장 높은 궤도를 **LUMO(Lowest Unoccupied Molecular Orbital)**은 비어있는 가장 낮은 궤도를 의미한다. 이때 HOMO와 LUMO 사이의 에너지 차이를 **Band Gap**이라 한다.

HOMO와 LUMO는 물질의 반응성과 산화환원 전위에 관련이 있다. Band Gap이 작으면 전자의 이동이 쉬워지기 때문에 반응성이 높아지고, Band Gap이 크면 전자의 이동이 어려워져 반응성이 낮아져 안정해진다. HOMO의 에너지가 높다면 전자를 잃기 쉽기 때문에 산화되기 쉽고, LUMO의 에너지가 낮다면 전자를 얻기 쉽기 때문에 환원되기 쉽다.

## Hückel 분자 궤도 이론

Hückel 분자 궤도 이론은 공액 파이 전자계의 파이 전자 분자 궤도 에너지를 계산하기 위해 원자 궤도의 선형 결합을 이용하는 이론으로, 이를 통해 공액계의 분자 궤도와 파이 전자의 에너지를 간단하게 예측할 수 있다. 이는 수학적으로 다음과 같이 표현된다.

$$E = \alpha + m \cdot \beta$$

이때  $\alpha$ 는 탄소 2p 궤도의 쿨롱 적분을,  $\beta$ 는 인접 탄소 간 공명 적분을,  $m$ 은 분자에 따라 달라지는 계수이다.

대표적으로 에틸렌은  $E_1 = \alpha + \beta$ ,  $E_2 = \alpha - \beta$ 의 형태로 표현되고, 부타디엔은  $E_1 = \alpha + 1.618... \beta$ ,  $E_2 = \alpha + 0.618... \beta$ ,  $E_3 = \alpha - 0.618... \beta$ ,  $E_4 = \alpha - 1.618 \beta$ 의 형태로 표현된다. 여기에서 1.618과 0.618은  $(\sqrt{5} \pm 1)/2$ 의 근사값이다.

## 밀도범함수 이론 (DFT, Density Functional Theory)

밀도범함수 이론(DFT)는 물질, 분자 내부에 전자가 들어있는 모양과 그 에너지를 양자역학적으로 계산하기 위한 이론으로, 이 이론을 통해 어떤 분자가 세상에 존재할 수 있는지에 대한 여부, 특정 분자의 모양과 성질 등을 예측할 수 있다. 이는 다전자계의 양자역학적 거동을 전자 밀도를 기반으로 계산한다.

슈뢰딩거 방정식을 직접 풀기 위해서는 전자 수가 N일 때 3N 차원 파동함수를 다뤄야 하는데, 전자의 개수가 많아지면 현실적으로 이를 푸는게 불가능해진다.

따라서 전자 각각의 움직임을 추적하는 것이 아니라 여러 전자를 한 덩어리로 취급하여 계산한다. 즉, 전자들을 각각의 파동함수로 보는 것이 아니라 **전자 밀도함수**로 대체하는 것이다.

전자의 밀도 함수는 다음과 같이 정의된다.

$$\rho(\vec{x}) = \sum_{i \in OCC} |\psi_i(\vec{x})|^2$$

DFT 계산을 위해서는 **범함수**와 **기저 함수**가 있어야 한다. DFT에서는 전자 밀도 함수를 입력받기 때문에 전자 밀도로부터 시스템의 총 에너지를 계산해주는 역할을 하는 함수가 범함수이다. 범함수에는 B3LYP, wB97X, PBE, M06-2X 등의 함수가 있다. 기저 함수는 분자를 구성하는 전자들의 파동 함수를 근사하여 표현하기 위해 사용되는 함수들의 집합으로 양자 화학 계산이 이론처럼 무한한 공간에서 이루어질 수 없기 때문에 유한한 함수들의 선형 결합을 통해 근사해야 한다. 따라서 기저 함수의 개수가 많아질수록 계산의 정확도는 높아지지만, 계산 비용과 시간이 증가하기 때문에 계산 목적에 맞는 기저 함수를 적절하게 선택해야 한다. 기저 함수에는 6-31G(d), G-311+G(d, p), def2-TZVP 등이 있다.

## 불포화도 (Degree of Unsaturation, DBE)

불포화도는 유기화합물 분자 구조가 수소가 얼마나 부족한지를 나타내는 척도로, 이중 결합, 삼중 결합, 고리 구조 등의 개수를 파악할 수 있다. 불포화도는 아래 공식을 통해 구할 수 있다.

$$DBE = C - \frac{H}{2} - \frac{X}{2} + \frac{N}{2} + 1$$

여기에서 C는 탄소 원자 수, H는 수소 원자 수, X는 할로젠 원자 수, N은 질소 원자 수를 의미한다.

이렇게 계산한 불포화도를 통해 이중결합, 삼중결합, 고리 구조의 개수의 합을 알아낼 수 있다.

$$DBE = \text{이중결합} + \text{삼중결합} + \text{고리구조}$$

## 그래프 동형

분자가 그래프의 형태로 표현된다면, 서로 다른 두 구조가 위상학적으로 같은지 확인하기 위해서는 두 그래프가 동형인지 파악해야 한다. 이를 위해 Canonical SMILES라는 방법을 통해 분자 구조를 유일한 문자열로 표현한 뒤 이를 비교하는 것으로 확인할 수 있다.

## 3. 이성질체 탐색

구조 이성질체를 탐색하기 위해 분자를 하나의 무방향 그래프로 생각하고, 그래프에서 노드를 원자, 간선을 결합, 가중치를 결합선의 개수로 생각한다면 이는 무방향 그래프에서 노드의 개수와 각 노드의 차수가 정해진 상태에서 그래프의 종류가 구조 이성질체의 개수가 될 것이다. 따라서 그래프를 재귀적으로 생성한 뒤, 그래프 동형 탐색을 통해 중복을 제거한다면 이성질체를 찾을 수 있다. 하지만 그래프 동형 검사는 매우 복잡하고 오래 걸리는 작업이고, 그래프를 재귀적으로 생성하게 된다면 제한 조건을 만족시키는 그래프를 얻기 위해 필요한 연산량이 매우 많아진다. 따라서 분자가 유기 화합물이라는 가정을 한 뒤, 탄소 화합물의 다양한 구조들과 불포화도를 바탕으로 이 과정을 최적화하려 한다.

## 알고리즘 설계

알고리즘의 전체적인 구조는 다음과 같다.

분자식 입력 (ex:  $C_4H_8$ )

1. 분자식 파싱
  2. 불포화도 계산
  3. 불포화도를 각 원자에 분배하는 경우의 수 계산
    - 탄소 골격 생성
    - 다중결합 배치
    - 원자가 전자 검증
    - 수소 추가
  4. Canonical SMILES 변환 및 중복 구조 제거
  5. DFT 계산 (psi4 라이브러리 활용)
- 이성질체 목록 + 안정성 분석 결과 출력 (에너지 준위 다이어그램 등)

## 분자식 파싱

정규표현식: `(([A-Z][a-z]?)(\d*))`

정규표현식을 통해 분자식 문자열에서 원자 종류와 개수를 추출한다.

예시

C4H8

- C: 4
- H: 8

## 불포화도 계산

이후 불포화도 공식을 통해 불포화도를 계산한다.

$$DBE = C - \frac{H}{2} - \frac{X}{2} + \frac{N}{2} + 1$$

예시

불포화도 =  $4 - 8/2 - 0 + 0 + 1 = 1$

## 불포화도 분배

불포화도를 계산하게 되면 다음과 같은 관계를 얻을 수 있다.

$$DBE = \text{이중결합} + \text{삼중결합} + \text{고리구조}$$

```
DBE = n_double + 2*n_triple + n_rings
# 제약조건
n_double >= 0
n_triple >= 0
n_rings >= 0
n_double, n_triple, n_rings은 모두 정수
```

따라서 각 결합의 경우의 수를 백트래킹 방식으로 탐색해볼 수 있다.

```
combinations = []

for n_triple in range(self.dbe // 2 + 1):
    remaining = self.dbe - 2 * n_triple

    for n_rings in range(remaining + 1):
        n_double = remaining - n_rings

        if n_double >= 0:
            combinations.append({
                'double': n_double,
                'triple': n_triple,
                'rings': n_rings
            })
```

n\_double과 n\_triple을 기준으로 반복문을 설정하여 탐색하면, DBE가 항상 같기 때문에 n\_double은 자동결정되는 방식이다. 이때 n\_double이 양수라면 제약조건을 만족하게 되므로, combinations에 해당 조합을 추가한다.

예시: DBE = 2

조합 1: double=2, triple=0, rings=0  
 조합 2: double=1, triple=0, rings=1  
 조합 3: double=0, triple=0, rings=2  
 조합 4: double=0, triple=1, rings=0

## 탄소 골격 생성

```
def generate_skeletons_with_rings(self, n_carbons, n_rings):
    """
    고리를 포함한 탄소 골격 생성
    """
    self.vprint(f"\n 골격 생성: C={n_carbons}, 고리={n_rings}")

    if n_rings == 0:
```

```

# 트리 구조
from networkx.generators.nonisomorphic_trees import nonisomorphic_trees
skeletons = list(nonisomorphic_trees(n_carbons))

valid_skeletons = []
for tree in skeletons:
    max_degree = max(dict(tree.degree()).values()) if tree.number_of_nodes() > 0 else 0
    if max_degree <= 4:
        for node in tree.nodes():
            tree.nodes[node]['element'] = 'C'
        valid_skeletons.append(tree)

self._vprint(f"   트리: {len(valid_skeletons)}개")
return valid_skeletons

elif n_rings == 1:
    # 단일 고리
    return self._generate_single_ring_skeletons(n_carbons)

else:
    # 다중 고리
    return self._generate_multi_ring_skeletons(n_carbons, n_rings)

```

구조에 고리가 없는 경우에는 `networkx.generators.nonisomorphic_trees` 라이브러리를 통해  $n$ 개 탄소의 모든 트리 구조를 생성하고, 탄소는 최대 4개의 결합이 가능하기 때문에 차수를 제한하여 차수 조건을 만족시키는 구조를 `valid_skeletons`에 모은다.

고리가 1개만 있다면 고리를 생성한 후 나머지 탄소를 부착하는 방식으로 생성한다. 그래프에서 고리 구조는 사이클로 볼 수 있기 때문에 `networkx` 라이브러리의 `nx.cycle_graph(ring_size)` 함수를 통해 사이클을 만들어주게 된다.

```

def _generate_single_ring_skeletons(self, n_carbons):
    """
    하나의 고리를 포함한 골격
    """
    skeletons = []

    # 최소 고리 크기는 3
    for ring_size in range(3, min(n_carbons + 1, 8)): # 큰 고리는 제한
        # 고리 생성
        ring = nx.cycle_graph(ring_size)

        # 원소 레이블
        for node in ring.nodes():
            ring.nodes[node]['element'] = 'C'

        remaining = n_carbons - ring_size

```

```

if remaining == 0:
    # 고리만
    skeletons.append(ring)
elif remaining > 0:
    # 나머지 탄소를 붙이기
    for attach_node in ring.nodes():
        if ring.degree(attach_node) < 4:
            g = ring.copy()

            # 체인 붙이기
            chain_start = max(g.nodes()) + 1
            g.add_edge(attach_node, chain_start)
            g.nodes[chain_start]['element'] = 'C'

            for i in range(1, remaining):
                prev = chain_start + i - 1
                curr = chain_start + i
                g.add_edge(prev, curr)
                g.nodes[curr]['element'] = 'C'

            skeletons.append(g)

self._vprint(f" 고리 골격: {len(skeletons)}개")
return skeletons

```

고리가 2개 이상 존재한다면 고리들의 크기로 가능한 조합을 우선적으로 탐색한다. 그런 다음, 각 고리들을 어떤 방식으로 연결할지를 탐색하고, 나머지 탄소를 분배하여 결합한다. 이때 고리 연결 방식은 독립형, 융합형, 가교형으로 나눌 수 있다.

#### 독립형이란?

독립형 연결 방식은 두 개 이상의 고리가 단일 탄소-탄소 결합으로 연결되어 있지만, 고리들이 서로 공통 원자를 공유하지는 않는 방식이다. 바이사이클로헥실과 같은 화합물에서 볼 수 있다.

#### 융합형이란?

융합형 연결 방식은 두 개의 고리가 두 개의 인접한 탄소 원자와 하나의 공유된 공유 결합을 공유하는 방식으로, 데칼린과 같은 화합물에서 볼 수 있는 구조이다.

#### 가교형이란?

가교형 연결 방식은 두 개의 고리가 두 개의 비인접한 탄소 원자를 공유하는 방식으로 노르보르난이 대표적이다.

아래 코드는 고리 크기 조합을 계산하는 함수이다. 고리의 크기는 최대 6으로 제한하였는데, 계산 시간이 너무 오래 걸리기도 하고, 큰 고리는 일반적으로 불안정하기 때문에 작은 고리에 대해서만 우선 탐색하도록 설정하였다. (추후에 이러한 성능 문제로 인해 발생한 탐색 범위 제약을 해결해나갈 것이다.)

```

def _generate_ring_size_combinations(self, n_carbons, n_rings):
    """
    n개 고리를 만들 수 있는 고리 크기 조합

```

```

"""
combos = []

# 최소 고리 크기 3, 최대 6
min_ring_size = 3
max_ring_size = 6

def backtrack(remaining_carbons, remaining_rings, current_combo):
    if remaining_rings == 0:
        if remaining_carbons >= 0:
            combos.append(current_combo[:])
        return

    # 융합 고리는 1개 탄소 공유, 스피로는 1개 공유
    # 보수적으로: 각 고리가 최소 3개 탄소 필요
    min_needed = remaining_rings * min_ring_size - (remaining_rings - 1)

    if remaining_carbons < min_needed:
        return

    for size in range(min_ring_size, min(max_ring_size + 1, remaining_carbons + 1)):
        current_combo.append(size)

        # 융합이면 1개 공유
        shared = 1 if len(current_combo) > 1 else 0

        backtrack(remaining_carbons - size + shared, remaining_rings - 1, current_combo)
        current_combo.pop()

backtrack(n_carbons, n_rings, [])

# 중복 제거 및 정렬
unique_combos = []
seen = set()

for combo in combos:
    key = tuple(sorted(combo))
    if key not in seen:
        seen.add(key)
        unique_combos.append(list(key))

return unique_combos

```

아래 코드는 여러 개의 고리가 존재할 때 구조를 계산하는 함수이다.

```

def _generate_multi_ring_skeletons(self, n_carbons, n_rings):
    """
    여러 개의 고리를 포함한 골격 생성

```

```

"""
skeletons = []

self._vprint(f"  다중 고리 생성: {n_rings}개 고리")

# 고리 크기 조합 생성
ring_size_combos = self._generate_ring_size_combinations(n_carbons, n_rings)

self._vprint(f"  가능한 고리 조합: {len(ring_size_combos)}가지")

for combo in ring_size_combos[:10]: # 처음 10개만
    self._vprint(f"    처리 중: {combo}")

    # 융합 고리
    fused = self._create_fused_rings(combo)
    if fused:
        skeletons.extend(fused)

    # 독립 고리
    spiro = self._create_spiro_rings(combo)
    if spiro:
        skeletons.extend(spiro)

    # 가교 고리
    if n_rings == 2: # 가교는 2개 고리일 때만
        bridged = self._create_bridged_rings(combo)
        if bridged:
            skeletons.extend(bridged)

self._vprint(f"  다중 고리 골격: {len(skeletons)}개")
return skeletons

```

각각의 고리 구조에 대한 계산은 `_create_fused_rings`, `_create_spiro_rings`, `_create_bridged_rings` 함수를 통해 이루어진다.

```

def _create_spiro_rings(self, ring_sizes):
    if len(ring_sizes) != 2:
        return []

    size1, size2 = ring_sizes
    skeletons = []

    ring1 = nx.cycle_graph(size1)

    for spiro_center in ring1.nodes():
        g = ring1.copy()

        start_node = max(g.nodes()) + 1

```

```

prev = spiro_center
for i in range(size2 - 1):
    curr = start_node + i
    g.add_edge(prev, curr)
    g.nodes[curr]['element'] = 'C'
    prev = curr

# 마지막 노드를 spiro_center에 연결
g.add_edge(prev, spiro_center)

skeletons.append(g)

return skeletons

```

```

def _create_fused_rings(self, ring_sizes):
    if len(ring_sizes) != 2:
        return []

    size1, size2 = ring_sizes
    skeletons = []

    ring1 = nx.cycle_graph(size1)

    for edge in ring1.edges():
        g = ring1.copy()

        u, v = edge

        if size2 == 3:
            # 삼각형 융합
            # u-v를 공유, 새 노드 1개 추가
            new_node = max(g.nodes()) + 1
            g.add_edge(u, new_node)
            g.add_edge(new_node, v)

        elif size2 == 4:
            # 사각형 융합
            # u-v를 공유, 새 노드 2개 추가
            n1 = max(g.nodes()) + 1
            n2 = n1 + 1
            g.add_edge(u, n1)
            g.add_edge(n1, n2)
            g.add_edge(n2, v)

        elif size2 == 5:
            # 오각형 융합

```

```

        n1 = max(g.nodes()) + 1
        n2 = n1 + 1
        n3 = n1 + 2
        g.add_edge(u, n1)
        g.add_edge(n1, n2)
        g.add_edge(n2, n3)
        g.add_edge(n3, v)

    elif size2 == 6:
        # 육각형 융합
        n1 = max(g.nodes()) + 1
        n2 = n1 + 1
        n3 = n1 + 2
        n4 = n1 + 3
        g.add_edge(u, n1)
        g.add_edge(n1, n2)
        g.add_edge(n2, n3)
        g.add_edge(n3, n4)
        g.add_edge(n4, v)

    else:
        continue

    for node in g.nodes():
        if 'element' not in g.nodes[node]:
            g.nodes[node]['element'] = 'C'

    skeletons.append(g)

return skeletons

```

```

def _create_bridged_rings(self, ring_sizes):
    if len(ring_sizes) != 2:
        return []

    size1, size2 = ring_sizes

    if size1 < 3 or size2 < 2:
        return []

    skeletons = []

    ring = nx.cycle_graph(size1)
    for node in ring.nodes():
        ring.nodes[node]['element'] = 'C'

    nodes = list(ring.nodes())

```

```

for i in range(len(nodes)):
    for j in range(i + 2, len(nodes)):
        if j - i < 2 or len(nodes) - j + i < 2:
            continue

        g = ring.copy()

        start = nodes[i]
        end = nodes[j]

        n_bridge_nodes = size2 - 2

        if n_bridge_nodes < 0:
            continue

        if n_bridge_nodes == 0:
            continue

        prev_node = start

        for k in range(n_bridge_nodes):
            new_node = max(g.nodes()) + 1
            g.add_node(new_node, element='C')

            g.add_edge(prev_node, new_node)

            prev_node = new_node

        g.add_edge(prev_node, end)

        if g.number_of_nodes() <= 15:
            skeletons.append(g)

return skeletons

```

## 다중결합 배치

앞에서 생성한 탄소 골격 구조를 바탕으로 골격에 이중결합과 삼중결합을 배치한다.

이중결합과 삼중결합에 대해 가능한 모든 위치를 순회하며 탐색한다.

```

def place_multiple_bonds(self, skeleton, n_double, n_triple):
    if n_double == 0 and n_triple == 0:
        # 다중결합 없음
        g = skeleton.copy()
        nx.set_edge_attributes(g, 1, 'bond_order')
        return [g]

```

```

structures = []
edges = list(skeleton.edges())

# 이중결합 위치 선택
if n_double > 0:
    for double_edges in itertools.combinations(edges, n_double):
        # 삼중결합 위치
        remaining = [e for e in edges if e not in double_edges]

        if n_triple > 0:
            for triple_edges in itertools.combinations(remaining, n_triple):
                g = self._create_multigraph(skeleton, double_edges, triple_edges)
                if g and self._check_valence(g):
                    structures.append(g)
            else:
                g = self._create_multigraph(skeleton, double_edges, [])
                if g and self._check_valence(g):
                    structures.append(g)
        else:
            # 삼중결합만
            for triple_edges in itertools.combinations(edges, n_triple):
                g = self._create_multigraph(skeleton, [], triple_edges)
                if g and self._check_valence(g):
                    structures.append(g)

return structures

```

이때 각 원소에 대해 원자가 전자 조건을 만족시키는지 검증하여 유효한 결합 배치만을 걸러낸다.

아래 코드가 각 원자에 대해 그래프에서 노드의 간선의 개수인 결합 차수를 바탕으로 결합 개수를 확인하여 원자가 전자의 개수가 올바른지 확인하는 코드이다. `valence_max`를 통해 C, O, N, H에서 가능한 최대 결합 수를 정의해 주었다.

```

def _check_valence(self, graph):
    valence_max = {'C': 4, 'O': 2, 'N': 3, 'H': 1}

    for node in graph.nodes():
        element = graph.nodes[node].get('element', 'C')
        max_val = valence_max.get(element, 4)

        # 총 결합 차수
        total_bonds = 0
        for neighbor in graph.neighbors(node):
            bond_order = graph.edges[node, neighbor].get('bond_order', 1)
            total_bonds += bond_order

```

```

    if total_bonds > max_val:
        return False

```

```

return True

```

이를 통해 화학적으로 전자 개수가 맞지 않아 불가능한 배치를 제외할 수 있었다.

## 수소 추가

결합 차수가 남아서 수소가 필요한 원자들에게 수소를 추가한다. 이를 위해 각 원자들에 대해 그래프에서의 결합 차수를 계산하고, 최대 결합 개수에서 이를 빼서 필요한 수소의 개수를 계산한다. 이후 수소들을 모두 추가한 뒤에 수소의 개수를 확인하고, 수소의 개수가 분자식으로부터 나온 수소의 개수와 일치하는지 검증한다.

```

def add_hydrogens(self, structures):
    completed = []
    valence_max = {'C': 4, 'O': 2, 'N': 3}

    for struct in structures:
        g = struct.copy()
        h_idx = max(g.nodes()) + 1 if g.number_of_nodes() > 0 else 0

        for node in list(g.nodes()):
            element = g.nodes[node].get('element', 'C')

            if element == 'H':
                continue

            # 현재 결합 차수 합
            total_bonds = 0
            for neighbor in g.neighbors(node):
                bond_order = g.edges[node, neighbor].get('bond_order', 1)
                total_bonds += bond_order

            # 필요한 수소
            max_val = valence_max.get(element, 4)
            needed_h = max_val - total_bonds

            # 수소 추가
            for _ in range(needed_h):
                g.add_node(h_idx, element='H')
                g.add_edge(node, h_idx)
                g.edges[node, h_idx]['bond_order'] = 1
                h_idx += 1

            # 수소 개수 확인
            h_count = sum(1 for n in g.nodes() if g.nodes[n]['element'] == 'H')
            expected_h = self.atom_counts.get('H', 0)

```

```

if h_count == expected_h:
    completed.append(g)

return completed

```

## SMILES 계산 및 중복 제거

생성한 이성질체 분자 구조를 SMILES 로 변환하고, 변환하여 생성된 문자열 배열에서 중복 원소를 제거하여 위상학적으로 같은 분자 구조를 제거한다.

```

unique_smiles = set()
for struct in with_hydrogen:
    smiles = self.graph_to_smiles(struct)
    if smiles:
        mol = Chem.MolFromSmiles(smiles)
        if mol:
            canonical = Chem.MolToSmiles(mol)
            unique_smiles.add(canonical)

self.isomers = sorted(list(unique_smiles))

```

## 밀도범함수 이론 (DFT)

생성한 이성질체 구조에 대한 안정도를 분석하기 위해 DFT를 사용하였다. DFT에서는 범함수와 기저 함수가 필요한데, 범함수의 경우 가장 널리 쓰이는 **B3LYP**를 사용하였고, 기저 함수는 **6-31G(d)**를 사용하였다.

기저 함수는 3-21G, 6-31G(d), 6-311G(d,p), def2-TZVP 등 다양하게 있지만, 정확도나 속도를 종합적으로 비교해보았을때 6-31G(d)가 정확도와 속도 측면에서 가장 적합할 것이라고 생각하여 이를 사용하였다.

```

class IsomerDFTAnalyzer:
    def __init__(self, smiles_list, method='B3LYP', basis='6-31G(d)', verbose=True):
        self.smiles_list = smiles_list
        self.method = method
        self.basis = basis

        self.results = []

        self.verbose = verbose

        # Psi4 설정
        psi4.set_memory('2 GB')
        psi4.set_num_threads(4)
        psi4.core.set_output_file('psi4_output.dat', False)

        print(f"DFT 설정: {method}/{basis}")

```

```

print(f"이성질체 개수: {len(smiles_list)}")

def _vprint(self, *args, **kwargs):
    if self.verbose:
        print(*args, **kwargs)

def smiles_to_psi4_geometry(self, smiles):
    """SMILES → 3D 좌표 → Psi4 geometry"""
    mol = Chem.MolFromSmiles(smiles)
    mol = Chem.AddHs(mol)

    # 3D 좌표 생성
    AllChem.EmbedMolecule(mol, randomSeed=42)
    AllChem.MMFFOptimizeMolecule(mol)

    conf = mol.GetConformer()
    atoms = mol.GetAtoms()

    geom_str = "O 1\n"

    for i, atom in enumerate(atoms):
        pos = conf.GetAtomPosition(i)
        symbol = atom.GetSymbol()
        geom_str += f"{symbol:2s} {pos.x:12.6f} {pos.y:12.6f} {pos.z:12.6f}\n"

    return geom_str

def calculate_single_isomer(self, smiles, optimize=True):
    self._vprint(f"\n계산 중: {smiles}")

    try:
        # Psi4 분자 객체
        geom_str = self.smiles_to_psi4_geometry(smiles)
        mol = psi4.geometry(geom_str)

        psi4.set_options({
            'basis': self.basis,
            'scf_type': 'df',
            'reference': 'rhf'
        })

        # 구조 최적화 + 에너지
        if optimize:
            energy, wfn = psi4.optimize(
                f'{self.method}/{self.basis}',
                molecule=mol,
                return_wfn=True # 파동 함수
            )

```

```

else:
    energy, wfn = psi4.energy(
        f'{self.method}/{self.basis}',
        molecule=mol,
        return_wfn=True
    )

# HOMO-LUMO
eps_a = wfn.epsilon_a() # Alpha 오비탈 에너지
homo_idx = wfn.nalpha() - 1
lumo_idx = wfn.nalpha()

eps_array = np.array([eps_a.get(i) for i in range(eps_a.dim())])

homo_energy = eps_array[homo_idx] * 27.211 # Hartree → eV
lumo_energy = eps_array[lumo_idx] * 27.211
gap = lumo_energy - homo_energy

# 쌍극자 모멘트
try:
    dipole_x = wfn.variable('CURRENT DIPOLE X')
    dipole_y = wfn.variable('CURRENT DIPOLE Y')
    dipole_z = wfn.variable('CURRENT DIPOLE Z')
    dipole_magnitude = np.sqrt(dipole_x**2 + dipole_y**2 + dipole_z**2)
except:
    dipole_magnitude = 0.0

result = {
    'smiles': smiles,
    'energy_hartree': energy,
    'energy_kcal': energy * 627.509,
    'homo_eV': homo_energy,
    'lumo_eV': lumo_energy,
    'gap_eV': gap,
    'dipole_debye': dipole_magnitude
}

self._vprint(f" ✓ Complete: E = {energy:.6f} Hartree")
self._vprint(f"   HOMO = {homo_energy:.3f} eV, LUMO = {lumo_energy:.3f} eV, Gap = {gap:.3f} eV")

return result

except Exception as e:
    self._vprint(f" × 오류: {str(e)}")
    return None

def calculate_all(self, optimize=True):

```

```

self._vprint(f"\n{' '*60}")
self._vprint(f" Start DFT Calculation")
self._vprint(f"{' '*60}\n")

self.results = []

for i, smiles in enumerate(self.smiles_list, 1):
    self._vprint(f"[{i}/{len(self.smiles_list)}]", end=" ")

    result = self.calculate_single_isomer(smiles, optimize)

    if result:
        self.results.append(result)

self._vprint(f"\n{' '*60}")
self._vprint(f" Calculation Complete: {len(self.results)}/{len(self.smiles_list)} success")
self._vprint(f"{' '*60}\n")

# 상대 에너지
if self.results:
    min_energy = min(r['energy_kcal'] for r in self.results)

    for result in self.results:
        result['relative_energy_kcal'] = result['energy_kcal'] - min_energy

return self.results

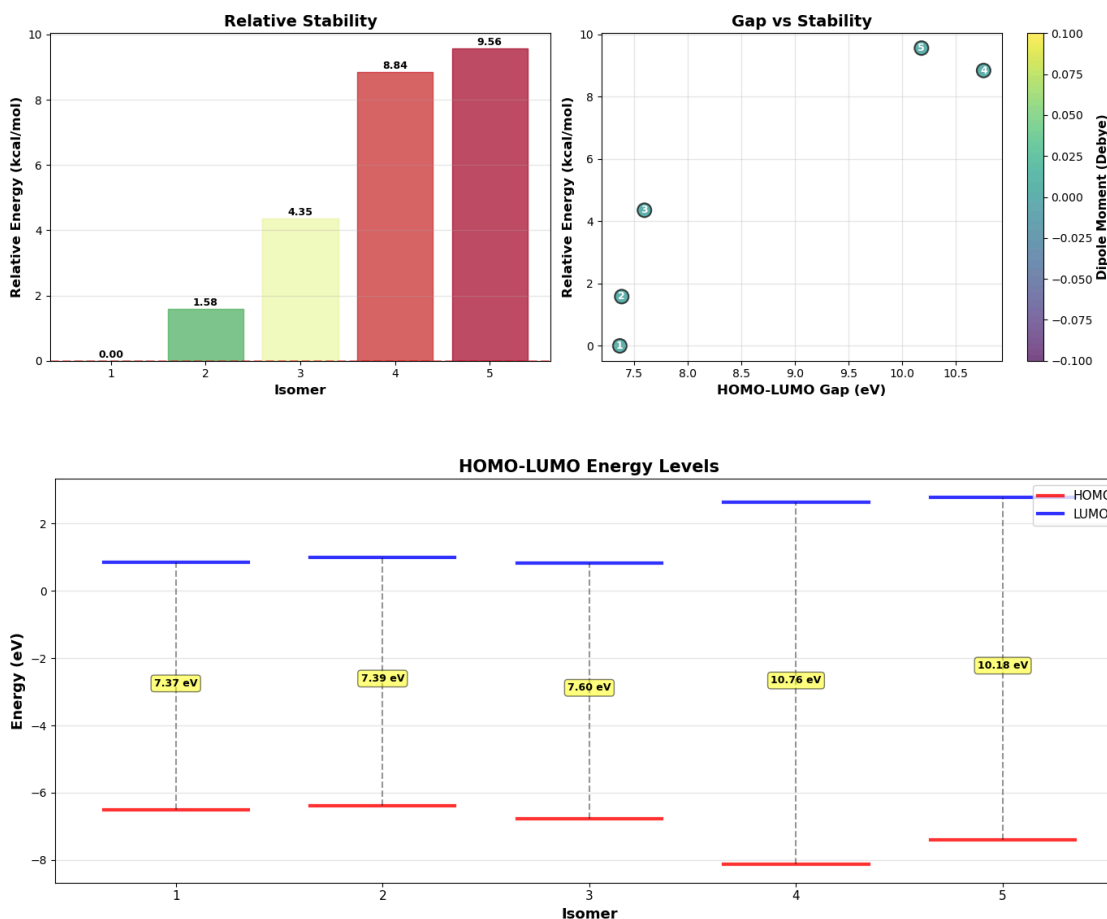
```

각 분자에 대해 몰당 에너지, HOMO, LUMO, Gap, 쌍극자 모멘트를 계산한다.

에너지의 경우 각각의 값을 보기보다는 여러 이성질체에서의 **상대적인** 안정도를 파악할 것이기 때문에 상대 에너지를 계산하여 분석하였다.

## 4. 결과

$C_4H_8$ 에 대한 구조 이성질체를 탐색해본 결과이다. 총 5개의 이성질체가 나왔고, 각각의 안정도와 에너지가 다른 것을 알 수 있다.



## 5. 결론

본 탐구에서는 그래프 이론과 양자화학 계산을 결합하여 구조 이성질체를 체계적으로 탐색하고 안정성을 평가하는 알고리즘을 성공적으로 개발하였다.

분자식으로부터 불포화도를 계산하고 이를 이중결합, 삼중결합, 고리 구조로 분배하는 조합을 백트래킹을 통해 가능한 모든 구조를 체계적으로 탐색할 수 있다. 또한 NetworkX 라이브러리를 통해 고리가 포함되었거나 포함되지 않은 구조의 생성을 구현함으로써 다양한 탄소 골격을 효율적으로 생성하였고, 이를 검증하기 위해 각 원자에 대해 원자가 전자와 결합 개수를 확인하였다. 생성한 이성질체 목록에서 원자가 전자 검증과 SMILES 변환을 통해 화학적으로 타당하지 않은 구조와 위상학적으로 중복되는 구조를 제거하여 탐색 결과의 정확성을 확보하였다.

이후 Psi4 라이브러리를 이용한 DFT 계산을 통해 각 이성질체의 에너지, HOMO, LUMO, Gap, 쌍극자 모멘트를 정량적으로 계산하고, 상대 에너지를 바탕으로 각 이성질체의 안정성을 비교/평가할 수 있었다.

## 6. 느낀점

이번 탐구를 진행하면서 화학 분야와 정보 기술의 융합을 통해 화학적 구조에 대한 연구가 가속화될 수 있겠다는 생각이 들었고, 화학의 이론적 내용을 바탕으로 발전한 DFT와 같은 이론들이 화학 연구에 어떠한 영향을 미치고 있는지 실감할 수 있었다.

또한 수업시간에 배운 불포화도나 분자 내에서의 원자간 상호작용에 관한 원리가 계산화학에서는 어떠한 방식으로 적용되어있는지 관련 최신 연구 동향을 알 수 있었다.

## 7. 향후 발전 방향

현재 알고리즘은 계산 시간 문제로 고리 크기를 6으로, 다중 고리 조합을 10가지로 제한하고 있다. 더 효율적인 그래프 생성 알고리즘을 도입하거나 병렬 처리를 활용하여 이러한 제약을 완화하고, 더 큰 분자와 복잡한 고리 시스템도 탐색할 수 있도록 개선이 필요할 것 같다.

또한 현재는 구조 이성질체만 다루고 있지만, 입체 이성질체(거울상 이성질체, 부분입체 이성질체, 기하 이성질체)까지 포함하면 훨씬 완전한 이성질체 탐색 시스템이 될 것 같다.

모든 이성질체에 대해 DFT 계산을 수행하는 것은 분자 크기가 커질수록 어려워진다. 기계학습 모델을 활용하여 안정적인 구조를 먼저 예측하고, 유력한 후보에 대해서만 정밀 DFT 계산을 수행하는 방식을 도입한다면 탐색의 효율성을 크게 높일 수 있을 것이다.

현재는 DFT만 사용하고 있지만, 더 정확한 결과가 필요한 경우를 위해 MP2, CCSD(T) 등의 고급 전자구조 이론을 선택적으로 사용할 수 있도록 확장하거나, 분산력 보정(D3, D4)을 추가하여 비공유 상호작용을 더 정확하게 다룰 수 있도록 개선해보면 좋을 것 같다.